Programació serveis i processos

Comunicació entre processos

Comunicació entre processos

Existeixen diverses formes per a comunicar dos processos en el sistema

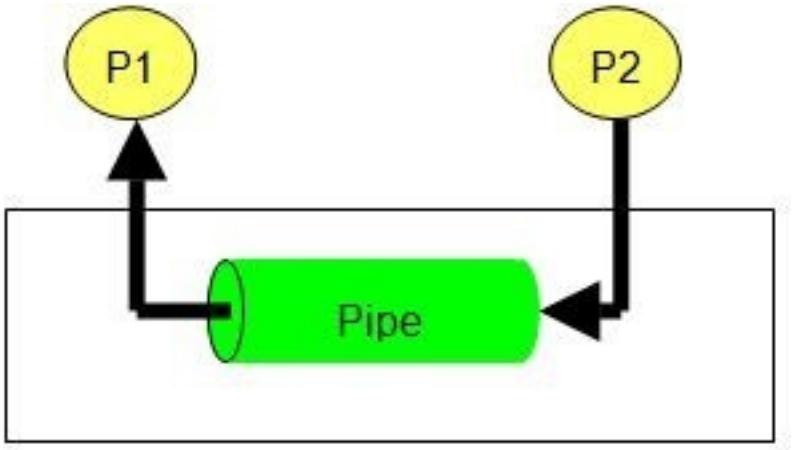
- PIPES
- Cues
- Semàfors
- Memòria compartida

PIPES

Fals fitxer.

Un procés escriu (P1)… un altre procés llig (P2)

Només entre pares i fills!



Utilitzar un PIPE

Creació

```
#include <uninstd.h>
int pipe (int fd[2])
```

La funció rep un array de dos elements (un per a lectura un altre per a escriptura).

De manera normal:

- fd[0] Conté descriptor lectura.
- fd[1] Conté descriptor escriptura.

Utilitzar un PIPE

Escriptura i lectura

*int read (int fd, void *buf, int count);*

*int write(int fd, void *buf, int count);*

Veurem millor la seua utilització mitjançant un exemple...

Exemple PIPE entre pare i fill

Veiem primera part del programa.

```
// EXEMPLE: FILL ENVIA MISSATGE AL PARE PER PIPE

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int fd[2]; //DECLARE VECTOR SENCERS
    char buffer[30];
    pid_t pid;

    pipe(fd); //CREE EL PIPE

    pid=fork(); //CREE PROCÉS FILL
```

Exemple PIPE entre pare i fill

```
switch(pid) {
    case -1: //ERROR
        printf("No s'ha pogut crear el fill\n");
        exit(-1);

    case 0: //FILL ESCRIU MISSATGE PEL DESCRIPTOR 1
        printf("El fill escriu al PIPE...\n");
        write (fd[1], "Hola papi" ,10);
        break;

    default: //PARE LLIG MISSATGE PEL DESCRIPTOR 0
        wait(NULL); //PARE ESPERA AL FILL ABANS DE LLEGIR
        printf("El pare llig del PIPE...\n");
        read (fd[0], buffer ,10);
        printf ("\tMissatge llegit:%s\n",buffer);
        break;
} //FI SWITCH
return 0;
```

Problema: Comunicació bidireccional

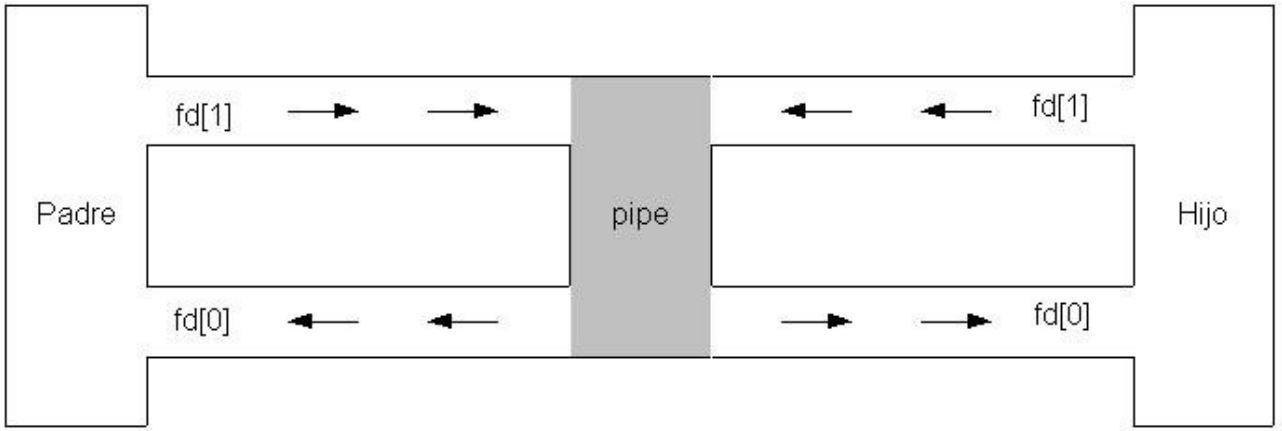
Al ser la comunicació en una única direcció hem de decidir en què direcció (de pare a fill o de fill a pare).

Descriptors es comparteixen entre processos pares i fills→ Hem de tancar extrem (no ens interessa fer-los servir).

Comunicació bidireccional

Flux informació de **pare a fill.**

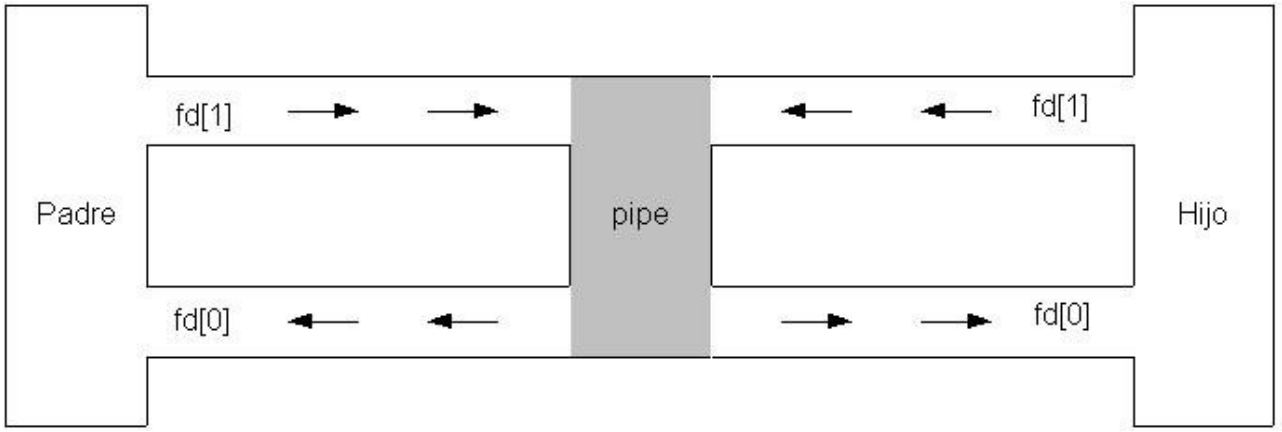
- El pare ha de tancar el descriptor de lectura fd[0] (no el farà servir)
- El fill ha de tancar el descriptor d'escriptura fd[1] (no el farà servir)



Comunicació bidireccional

Flux informació de **fill a pare.**

- El pare ha de tancar el descriptor de d'escriptura fd[1] (no el farà servir)
- El fill ha de tancar el descriptor de lectura fd[0] (no el farà servir)



Codi comunicació pare a fill.

Primera part igual que l'anterior....

```
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char saludoPadre[]="Bon dia, fill.\0";
    char buffer[25];

    int fd[2]; //DECLARE VECTOR SENCERS
    pipe(fd); //CREE EL PIPE

    pid_t pid;
    pid=fork(); //CREE PROCÉS FILL
```

Codi comunicació pare a fill.

Observa com respecta les normes anteriors

```
switch(pid) {
    case -1: //ERROR
        printf("No s'ha pogut crear el fill\n");
        exit(-1);

    case 0: //FILL LLIG MISSATGE PEL DESCRIPTOR 0
        close (fd[1]); //TANCA EL DESCRIPTOR D'EIXIDA
        printf ("El fill llig del PIPE...\n", buffer);
        read (fd[0],buffer, sizeof(buffer));
        printf ("\tMissatge llegit:%s\n", buffer);
        break;
    default: //PARE ESCRIU MISSATGE PEL DESCRIPTOR 1
        close (fd[0]); //TANCA EL DESCRIPTOR D'ENTRADA
        printf ("\El pare escriu al PIPE%s\n",buffer);
        write (fd[1],saludoPadre, strlen(saludoPadre));
        wait(NULL); //PARE ESPERA AL FILL
        break;
}
```

PIPES amb noms : FIFOS

Els FIFOS permeten comunicar processos que no siguen pare i fill.

Un FIFO és com un fitxer amb nom que existeix en el sistema de fitxers i que poden obrir, llegir i escriure múltiples processos.

Les dades escrites es lligen com en una cua, i una vegada llegits no poden ser llegits de nou.

Característiques FIFO

- Una operació d'escriptura en un FIFO queda en espera fins que el procés pertinent òbriga el FIFO per a iniciar la lectura.

- Només es permet l'escriptura d'informació quan un procés vaja a recollir aquesta informació.

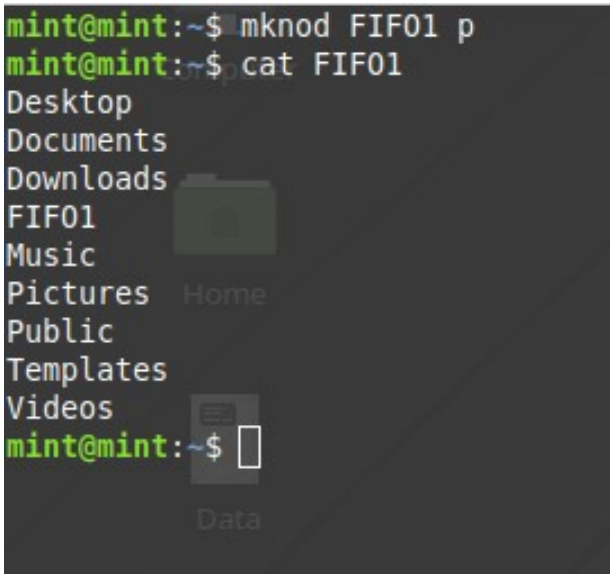
Utilització FIFO: funció mknod

Es pot executar des de terminal:

En un terminal:

$ mknod FIFO1 p //cree FIFO

$ cat FIFO1 //en espera fins que entre alguna cosa..



En un altre terminal:

$ ls>FIFO1

.. s'executa el cat.

mknod en C

Funció:

mknod("FIFO1", S_IFIFO|0644, 0);

- "FIFO1" es crea com a arxiu FIFO.
- S_IFIFO: mode de creació. Crea un FIFO (amb S_IFIFO) i permisos 644.
- 0 (tercer argument): s'ignora (en crear un FIFO)

Exemple: crea i llig cadena de FIFO

Crea el FIFO, ho obri i llig cadena (si la conté).

```
mknod("FIFO1",S_IFIFO|0660,0); //CREA FIFO

while(TRUE) { //OBRE, LLIG, TANCA FIFO ETERNAMENT
    fp=open("FIFO1",O_RDONLY); //OBRE FIFO
    nbytes=read(fp,buffer,TAM_BUF-1); //LLIG DE LA FIFO
    buffer[nbytes]='\0'; //CARÀCTER FINAL CADENA EN C
    printf("Cadena rebuda: %s \n",buffer);
    close(fp); //TANCA FIFO
}
```

Exemple: escriu en FIFO

Obri arxiu per a escriptura, escriu i tanca

```
int main()
{
    int fp;
    char salutacio[]="Salutacions!!!";
    fp=open("FIFO1",1); // OBRE LA FIFO

    if (fp==-1) {
        printf ("Error a l'obrir l'arxiu\n");
        exit(1);
    }
    printf ("Enviant informació al FIFO...\n");
    write (fp,salutacio,strlen(salutacio));
    close(fp); // TANQUE LA FIFO
    return 0;
}
```